# BÁO CÁO ĐỒ ÁN 1

## Ma Trận và Cơ Sở của Tính Toán Khoa Học

Tháng 4 năm 2026

**Thành viên nhóm:**
Nguyễn Duy Vũ - MSSV: 24120495
Phạm Văn Hữu Tài - MSSV: 24120435
Hà Lê Thanh Thúy - MSSV: 24120461
Trần Minh Tiến - MSSV: 24120465
Nguyễn Hữu Tường Vân - MSSV: 24120489

Trường Đại học Khoa học Tự nhiên
Khoa Công nghệ Thông tin
Năm: 2026
*Nhóm 10 - Lớp: CTT3*

***Giảng viên hướng dẫn:*** *Lê Nhựt Nam, Võ Nam Thục Đoan*

# Mục lục


1. Phần 1: Phép khử Gauss và Các Ứng Dụng . . . . . 5
   1.1. Định nghĩa và Cơ sở toán học . . . . . 5
      1.1.1. Phương pháp khử Gauss và Gauss–Jordan . . . . . 5
      1.1.2. Khử Gauss có chọn phần tử chốt (Partial Pivoting) . . . . . 6
      1.1.3. Tính Định Thức qua Khử Gauss . . . . . 6
      1.1.4. Ma Trận Nghịch Đảo bằng Gauss–Jordan . . . . . 6
      1.1.5. Hạng Ma Trận và Cơ Sở . . . . . 6
   1.2. Thuật Toán . . . . . 6
   1.3. Cài Đặt Yêu Cầu . . . . . 9
      1.3.1. Cấu trúc thư mục . . . . . 10
      1.3.2. Cài đặt . . . . . 10
   1.4. Tiêu Chí Đánh Giá — Phần 1 . . . . . 10
2. Phần 2: Phân Rã Ma Trận và Trực Quan Hóa với Manim . . . . . 11
   2.1. Chéo Hóa Ma Trận (Diagonalizable Matrix) . . . . . 11
      2.1.1. Định nghĩa và Cơ sở toán học . . . . . 11
      2.1.2. Điều kiện để chéo hóa Ma trận . . . . . 11
      2.1.3. Các bước giải tay (Manual Steps) . . . . . 12
   2.2. Thiết kế và cài đặt thuật toán phân rã QR . . . . . 12
      2.2.1. Phân rã QR bằng Gram-Schmidt . . . . . 12
      2.2.2. Thuật toán QR tìm giá trị riêng . . . . . 13
      2.2.3. Giải vector riêng bằng RREF . . . . . 14
      2.2.4. Hàm chéo hóa tổng quát . . . . . 14
      2.2.5. Kết quả thực nghiệm . . . . . 15
   2.3. Kỹ Thuật Phân Rã SVD . . . . . 17
      2.3.1. Cơ sở toán học . . . . . 17
      2.3.2. Thiết kế và cài đặt thuật toán phân rã SVD . . . . . 17
      2.3.3. Kiến trúc thuật toán trong chương trình . . . . . 18
      2.3.4. Đánh giá . . . . . 19
   2.4. Trực Quan Hóa với Manim . . . . . 20
      2.4.1. Phân rã Giá trị Kỳ dị (SVD) từng bước . . . . . 20
      2.4.2. Trực quan hóa Biến đổi Hình học . . . . . 21
      2.4.3. Quá trình thực hiện Chéo hóa (Diagonalization) . . . . . 21
   2.5. Cài đặt và Kết quả thực nghiệm . . . . . 22
      2.5.1. Tổ chức cấu trúc dữ liệu . . . . . 22
      2.5.2. Đánh giá độ chính xác và Sức chịu tải (Stress Test) . . . . . 22
   2.6. Tiêu chí đánh giá — Phần 2 . . . . . 22
3. Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng . . . . . 23
   3.1. Giới thiệu . . . . . 23
      3.1.1. Mục tiêu cụ thể . . . . . 23

3.1.2. Cấu trúc thư mục ..... 23
3.2. Cơ sở lý thuyết ..... 23
3.2.1. Số điều kiện (Condition Number) ..... 23
3.2.2. Phương pháp Khử Gauss với Partial Pivoting ..... 24
3.2.3. Phương pháp SVD (Singular Value Decomposition) ..... 24
3.2.4. Phương pháp lặp Gauss-Seidel ..... 24
3.2.5. Bảng so sánh tổng quan ..... 24
3.3. Phương pháp thực nghiệm ..... 25
3.3.1. Ma trận thí nghiệm ..... 25
3.3.2. Phương pháp đo ..... 25
3.4. Phân tích hiệu năng (Performance Analysis) ..... 25
3.4.1. Kết quả đo thời gian thực thi ..... 25
3.4.2. Phân tích đồ thị Log-Log và Độ phức tạp ..... 26
3.5. Phân tích tính ổn định số (Numerical Stability) ..... 26
3.5.1. Kết quả trên ma trận Hilbert (Ill-conditioned) ..... 26
3.5.2. Kết quả trên ma trận SPD ngẫu nhiên (Well-conditioned) ..... 26
3.5.3. Nhận xét và phân tích ..... 26
3.5.4. Mối quan hệ $\kappa(A)$ và sai số ..... 27
3.6. Kết luận và Khuyến nghị ..... 27
3.7. Tiêu chí đánh giá — Phần 3 ..... 27
4. Kết luận ..... 28
4.1. Tóm tắt kết quả đạt được ..... 28
4.2. Bài học rút ra ..... 28
4.3. Khó khăn gặp phải ..... 28
5. Bảng phân công công việc ..... 29
Tài Liệu Tham Khảo ..... 30
Các nguồn tham khảo trực tuyến: ..... 30

# Giới thiệu Đồ án

**Mục tiêu tổng quát**

Đồ án này tập trung vào ba nhóm kỹ thuật cốt lõi của đại số tuyến tính số trên máy tính:

1. **Phép khử Gauss:** Nền tảng giải hệ phương trình tuyến tính, tính định thức, nghịch đảo và hạng ma trận.
2. **Phân rã ma trận và Chéo hóa:** Hiểu bản chất hình học và trực quan hóa bằng Manim.
3. **Ổn định số học và chi phí tính toán:** Đánh giá thực nghiệm các phương pháp số (Phần điểm cộng).

Sinh viên tự cài đặt từ đầu (from scratch) bằng Python, trực quan hóa bằng Manim, và phân tích thực nghiệm.

Tuyệt đối không dùng các hàm giải có sẵn như `numpy.linalg.solve` hay `scipy.linalg.lu` cho phần thuật toán lõi.

# 1. Phần 1: Phép khử Gauss và Các Ứng Dụng

**Tóm tắt yêu cầu Phần 1**

Cài đặt phép khử Gauss có partial pivoting từ đầu bằng Python, rồi sử dụng để: **(1)** giải hệ phương trình tuyến tính, **(2)** tính định thức, **(3)** tìm ma trận nghịch đảo, **(4)** tính hạng và tìm cơ sở.

## 1.1. Định nghĩa và Cơ sở toán học

### 1.1.1. Phương pháp khử Gauss và Gauss–Jordan

Xét hệ phương trình tuyến tính $\boldsymbol{Ax} = \boldsymbol{b}$ với $\boldsymbol{A} \in \mathbb{R}^{m\times n}$, $\boldsymbol{x} \in \mathbb{R}^n$, $\boldsymbol{b} \in \mathbb{R}^m$.

**Định nghĩa 1.1** (Ma trận tăng cường). Ma trận tăng cường (augmented matrix) của hệ $\boldsymbol{Ax} = \boldsymbol{b}$ là:

$$[\boldsymbol{A} \mid \boldsymbol{b}] = \left(\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} & b_m \end{array}\right) \tag{1}$$

**Định nghĩa 1.2** (Các phép biến đổi dòng sơ cấp). Ba phép biến đổi dòng sơ cấp (elementary row operations) trên ma trận:
1. $R_i \leftrightarrow R_j$: Hoán đổi dòng $i$ và dòng $j$.
2. $R_i \leftarrow c \cdot R_i$, với $c \neq 0$: Nhân dòng $i$ với hằng số $c$.
3. $R_i \leftarrow R_i + c \cdot R_j$: Cộng $c$ lần dòng $j$ vào dòng $i$.

Ba phép biến đổi này **không làm thay đổi tập nghiệm** của hệ phương trình.

**Định nghĩa 1.3** (Dạng bậc thang dòng — REF). Ma trận $\boldsymbol{U}$ ở dạng bậc thang dòng (Row Echelon Form, viết tắt REF) nếu:
1. Mọi dòng toàn số 0 nằm phía dưới các dòng khác 0.
2. Phần tử chỉ huy (pivot) của mỗi dòng khác 0 nằm bên phải pivot của dòng ngay trên nó.

**Định nghĩa 1.4** (Dạng bậc thang dòng rút gọn — RREF). Ma trận $\boldsymbol{R}$ ở dạng bậc thang dòng rút gọn (Reduced REF, viết tắt RREF) nếu ngoài điều kiện REF còn có:
3. Mỗi pivot bằng 1.
4. Mỗi cột chứa pivot có tất cả các phần tử còn lại bằng 0.

### 1.1.2. Khử Gauss có chọn phần tử chốt (Partial Pivoting)

Trong cài đặt thực tế, cần áp dụng khử Gauss có chọn phần tử chốt hay partial pivoting để đảm bảo tính ổn định số học.

**Định lý 1.1** (Partial Pivoting). Tại bước khử thứ $k$, chọn hàng $p$ sao cho:

$$|a_{pk}^{(k)}| = \max_{k \le i \le n} |a_{ik}^{(k)}| \tag{2}$$

rồi hoán đổi dòng $k$ và dòng $p$ trước khi thực hiện khử. Kỹ thuật này giảm thiểu đáng kể sai số làm tròn (round-off error) phát sinh trong tính toán dấu phẩy động.

### 1.1.3. Tính Định Thức qua Khử Gauss

Sau khi đưa ma trận $\boldsymbol{A}$ về dạng tam giác trên $\boldsymbol{U}$ bằng phép khử Gauss với $s$ lần hoán đổi dòng, định thức được tính theo công thức:

$$\det(\boldsymbol{A}) = (-1)^s \cdot \prod_{i=1}^{n} u_{ii}. \tag{3}$$

### 1.1.4. Ma Trận Nghịch Đảo bằng Gauss–Jordan

**Định lý 1.2.** Ma trận $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ khả nghịch khi và chỉ khi $\det(\boldsymbol{A}) \neq 0$. Nghịch đảo $\boldsymbol{A}^{-1}$ được tìm bằng cách thực hiện biến đổi dòng đồng thời trên ma trận ghép $[\boldsymbol{A} \mid \boldsymbol{I}_n]$ cho đến khi đạt dạng $[\boldsymbol{I}_n \mid \boldsymbol{A}^{-1}]$.

### 1.1.5. Hạng Ma Trận và Cơ Sở

**Định nghĩa 1.5** (Hạng ma trận). Hạng (rank) của ma trận $\boldsymbol{A}$, ký hiệu $\operatorname{rank}(\boldsymbol{A})$), bằng số dòng khác 0 trong dạng REF của $\boldsymbol{A}$, hoặc tương đương, bằng số cột pivot.

Từ dạng RREF, ta xác định được ba không gian con quan trọng:

- **Không gian cột** (column space) $\mathcal{C}(\boldsymbol{A})$: sinh bởi các cột pivot của $\boldsymbol{A}$ gốc.
- **Không gian dòng** (row space) $\mathcal{R}(\boldsymbol{A})$: sinh bởi các dòng khác 0 trong RREF.
- **Không gian nghiệm** (null space) $\mathcal{N}(\boldsymbol{A})$: tập nghiệm của $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{0}$.

## 1.2. Thuật Toán

**Thuật toán 1.1** (Phép khử Gauss với Partial Pivoting).

**Đầu vào:** Ma trận $\boldsymbol{A} \in \mathbb{R}^{n \times m}$, vector $\boldsymbol{b} \in \mathbb{R}^n$.

**Đầu ra:** Nghiệm $\boldsymbol{x}$, số lần hoán đổi $s$, danh sách chỉ số pivot.

1. Tạo ma trận tăng cường $M = [\boldsymbol{A} \mid \boldsymbol{b}]$, đặt $s = 0$.
2. Với $k = 1, 2, ..., n$:

   a. Tìm $p = \arg\max_{k \le i \le n} |M_{ik}|$. Nếu $|M_{pk}| = 0$: báo lỗi không tồn tại pivot tại cột $k$ (hệ không có nghiệm duy nhất); nếu $|M_{pk}| < \varepsilon$: cảnh báo pivot gần bằng 0, hệ có thể không ổn định số học (ill-conditioned).

   b. Nếu $p \neq k$: hoán đổi dòng $k \leftrightarrow p$, tăng $s \leftarrow s + 1$.

   c. Với $i = k + 1, ..., n$: tính nhân tử $l_{ik} = \frac{M_{ik}}{M_{kk}}$, cập nhật $M_{i,*} \leftarrow M_{i,*} - l_{ik} \cdot M_{k,*}$.
3. **Thế ngược:** Giải hệ tam giác trên $\boldsymbol{U}\boldsymbol{x} = \boldsymbol{c}$ từ dưới lên:

$$x_i = \frac{1}{M_{ii}} \left( M_{i,m+1} - \sum_{j=i+1}^{n} M_{ij} x_j \right), \quad i = n, n-1, ..., 1. \tag{4}$$

Sinh viên cần đưa ra công thức nghiệm tổng quát trong trường hợp vô số nghiệm.

**Giải pháp của nhóm:** Nhóm đã bổ sung hàm `general_solution` để trích xuất không gian nghiệm (Nullspace) trong trường hợp hệ vô số nghiệm bằng cách đưa ma trận về dạng RREF, trích xuất nghiệm riêng $x_p$ và các vector cơ sở $v_i$ tương ứng với các biến tự do.

**Thuật toán 1.2** (Tính định thức bằng phương pháp khử Gauss và chọn phần tử chốt).

**Đầu vào:** Ma trận vuông $\boldsymbol{A} \in \mathbb{R}^{n \times n}$. **Đầu ra:** Giá trị định thức $\det(\boldsymbol{A})$.

1. Khởi tạo giá trị định thức $D = 1.0$, biến dấu $\sigma = 1$, và sai số $\varepsilon = 10^{-12}$. Sao chép $\boldsymbol{M} \leftarrow \boldsymbol{A}$.
2. Với mỗi cột $k$ từ 1 đến $n$:

   a. **Chọn chốt:** Tìm dòng $p$ ($k \le p \le n$) sao cho $|M_{pk}|$ đạt cực đại.

   b. **Kiểm tra:** Nếu $|M_{pk}| < \varepsilon$, trả về $\det(\boldsymbol{A}) = 0$.

   c. **Hoán đổi:** Nếu $p \neq k$: hoán đổi dòng $\boldsymbol{M}_k \leftrightarrow \boldsymbol{M}_p$ và đặt $\sigma \leftarrow \sigma \cdot (-1)$.

   d. **Tích lũy:** $D \leftarrow D \cdot M_{kk}$.

   e. **Khử:** Với mỗi dòng $i$ từ $k + 1$ đến $n$:
   - $l_{ik} = \frac{M_{ik}}{M_{kk}}$.
   - $\boldsymbol{M}_i \leftarrow \boldsymbol{M}_i - l_{ik} \cdot \boldsymbol{M}_k$.

3. **Kết quả:** Trả về $\sigma \cdot D$.

**Thuật toán 1.3** (Tìm ma trận nghịch đảo bằng Gauss-Jordan).

Để tìm ma trận nghịch đảo $\boldsymbol{A}^{-1}$, nhóm sử dụng thuật toán Gauss-Jordan trên ma trận mở rộng. Phương pháp này biến đổi song song ma trận $\boldsymbol{A}$ và ma trận đơn vị $\boldsymbol{I}$ để thu được kết quả trực tiếp.

**Đầu vào:** Ma trận vuông $\boldsymbol{A} \in \mathbb{R}^{n \times n}$. **Đầu ra:** Ma trận nghịch đảo $\boldsymbol{A}^{-1}$ hoặc báo lỗi nếu ma trận suy biến.

1. **Khởi tạo:** Ghép ma trận đơn vị $\boldsymbol{I}_n$ vào bên phải ma trận $\boldsymbol{A}$ để tạo ma trận mở rộng $\boldsymbol{M} = [\boldsymbol{A} \mid \boldsymbol{I}]$ kích thước $n \times 2n$.
2. Với mỗi cột $k$ từ 1 đến $n$:

   a. **Chọn phần tử chốt (Partial Pivoting):** Tìm dòng $p$ $(k \leq p \leq n)$ sao cho $|M_{pk}|$ lớn nhất. Nếu $|M_{pk}| < \varepsilon$, báo lỗi ma trận suy biến và dừng thuật toán.

   b. **Hoán đổi dòng:** Nếu $p \neq k$, hoán đổi dòng $\boldsymbol{M}_k \leftrightarrow \boldsymbol{M}_p$.

   c. **Chuẩn hóa dòng chốt:** Chia toàn bộ dòng $\boldsymbol{M}_k$ cho phần tử chốt $M_{kk}$ để đưa phần tử tại vị trí $(k, k)$ về bằng 1.0.

   d. **Khử Gauss-Jordan:** Với mọi dòng $i$ từ 1 đến $n$ (ngoại trừ dòng $k$):
   - Tính nhân tử $f = M_{ik}$.
   - Cập nhật dòng: $\boldsymbol{M}_i \leftarrow \boldsymbol{M}_i - f \cdot \boldsymbol{M}_k$.
3. **Trích xuất:** Ma trận nghịch đảo $\boldsymbol{A}^{-1}$ chính là nửa bên phải (các cột từ $n+1$ đến $2n$) của ma trận $\boldsymbol{M}$.

**Phân tích kỹ thuật của nhóm:**

- Khác với khử Gauss thông thường, thuật toán Gauss-Jordan triệt tiêu các phần tử ở cả phía trên và phía dưới chốt, giúp đưa vế trái về ma trận đơn vị mà không cần bước thế ngược.
- Nhóm đã thực hiện kiểm thử trên các ma trận có phần tử $M_{00} = 0$ (Test 2) để chứng minh tầm quan trọng của việc hoán đổi dòng nhằm tránh lỗi chia cho 0.
- Độ chính xác được kiểm chứng qua biểu thức $\left\|\boldsymbol{A}\boldsymbol{A}^{-1} - \boldsymbol{I}\right\|_\infty$, đạt sai số cực thấp ($\approx 10^{-16}$).

Đây là ứng dụng tổng hợp của phép khử Gauss-Jordan. Bằng cách đưa ma trận về dạng hàng bậc thang rút gọn (RREF), ta có thể xác định được hạng của ma trận và cơ sở của ba không gian vector quan trọng: không gian cột, không gian dòng và không gian nghiệm.

**Thuật toán 1.4** (Xác định hạng và các không gian vector cơ bản).

**Đầu vào:** Ma trận bất kỳ $\boldsymbol{A} \in \mathbb{R}^{m \times n}$.

**Đầu ra:** Hạng $r$, cơ sở không gian cột $C(\boldsymbol{A})$, không gian dòng $R(\boldsymbol{A})$, và không gian nghiệm $N(\boldsymbol{A})$.

1. **Biến đổi RREF:** Sử dụng thuật toán Gauss-Jordan để đưa $\boldsymbol{A}$ về dạng hàng bậc thang rút gọn $\boldsymbol{R}$. Lưu lại tập hợp chỉ số các cột chứa phần tử chốt $J_{\text{pivot}}$.
2. **Xác định Hạng (Rank):** $r = |J_{\text{pivot}}|$ (số lượng cột chứa chốt).
3. **Xác định các cơ sở (Basis):**
   - **Không gian cột ($C(\boldsymbol{A})$):** Lấy các cột tương ứng với chỉ số $J_{\text{pivot}}$ từ ma trận gốc $\boldsymbol{A}$.
   - **Không gian dòng ($R(\boldsymbol{A})$):** Lấy các dòng khác không từ ma trận $\boldsymbol{R}$.
   - **Không gian nghiệm ($N(\boldsymbol{A})$):** Với mỗi biến tự do (cột không chứa chốt), thiết lập vector cơ sở bằng cách giải hệ $\boldsymbol{R}\boldsymbol{x} = \boldsymbol{0}$.

**Phân tích và Kiểm chứng định lý (Nhóm 10):**

- **Định lý Rank-Nullity:** Nhóm đã kiểm chứng thực nghiệm trên các ma trận đặc biệt (ma trận toàn không, ma trận đứng), kết quả luôn thỏa mãn: $\text{rank}(\boldsymbol{A}) + \text{nullity}(\boldsymbol{A}) = n$.
- **Tính toán cơ sở:** Khác với không gian dòng, cơ sở không gian cột phải được trích xuất từ ma trận gốc $\boldsymbol{A}$ để đảm bảo tính chính xác về không gian vector ban đầu.
- **Độ tin cậy:** Các vector trong không gian nghiệm được kiểm chứng bằng phép nhân $\boldsymbol{A} \cdot \boldsymbol{v} \approx \boldsymbol{0}$ với sai số đạt ngưỡng $10^{-16}$.

## 1.3. Cài Đặt Yêu Cầu

**Yêu cầu cài đặt — Phần 1**

Sinh viên cài đặt **từ đầu** (không dùng NumPy/SciPy/SymPy) các hàm sau:

1. `gaussian_eliminate(A, b)` — Trả về ma trận sau khi khử, nghiệm $\boldsymbol{x}$, số lần hoán đổi.

2. `back_substitution(U, c)` — Giải hệ tam giác trên.
3. `determinant(A)` — Tính $\det(\boldsymbol{A})$ qua khử Gauss.
4. `inverse(A)` — Tính $\boldsymbol{A}^{-1}$ bằng phương pháp Gauss–Jordan.
5. `rank_and_basis(A)` — Tính hạng và cơ sở của các không gian cột, dòng, nghiệm.
6. `verify_solution(A, x, b)` — Kiểm chứng kết quả bằng NumPy.

### 1.3.1. Cấu trúc thư mục

```
part1/
├── gaussian.py
├── determinant.py
├── inverse.py
├── rank_basic.py
└── part1_demo.ipynb
```

### 1.3.2. Cài đặt

**Đánh giá kết quả cài đặt:** Nhóm đã cài đặt hoàn chỉnh 6 hàm trên. Đồng thời bổ sung bộ kiểm thử gồm **6 Test Cases** (bao gồm ma trận suy biến, zero pivot, ill-conditioned và stress test $100 \times 100$) để chứng minh tính đúng đắn. Tất cả các test đều khớp với NumPy với sai số $\leq 10^{-12}$.

## 1.4. Tiêu Chí Đánh Giá — Phần 1

| Tiêu chí | Mô tả | Điểm |
|---|---|---|
| Cài đặt Gauss cơ bản | Chạy đúng, có kiểm thử rõ ràng | 1.5 |
| Partial Pivoting | Xử lý đúng khi pivot nhỏ hoặc bằng 0 | 0.5 |
| Tính định thức | Đúng dấu và giá trị | 0.5 |
| Ma trận nghịch đảo | Gauss–Jordan, kiểm thử $\boldsymbol{A}\boldsymbol{A}^{-1} = \boldsymbol{I}$ | 1.0 |
| Hạng và cơ sở | Đúng rank, trả về cơ sở các không gian | 1.0 |
| Kiểm chứng | So sánh kết quả với NumPy/SciPy | 0.5 |
| Tổng Phần 1 | **Đã hoàn thành 100%** | **5.0** |

# 2. Phần 2: Phân Rã Ma Trận và Trực Quan Hóa với Manim

**Tóm tắt yêu cầu Phần 2**

Sinh viên **chọn một kỹ thuật** phân rã ma trận (QR, LU, SVD hoặc Cholesky), cài đặt bằng Python, và **tạo video demo** bằng Manim trực quan hóa quá trình phân rã cùng với chéo hóa ma trận.

## 2.1. Chéo Hóa Ma Trận (Diagonalizable Matrix)

**Tổng quan về Chéo hóa**

Chéo hóa ma trận là một phép biến đổi tuyến tính nhằm đưa một ma trận vuông phức tạp về dạng ma trận đường chéo đơn giản. Quá trình này không chỉ làm rõ các đặc trưng hình học của ma trận mà còn giúp tối ưu hóa các phép tính lũy thừa và giải hệ phương trình vi phân.

### 2.1.1. Định nghĩa và Cơ sở toán học

Ma trận $A \in \mathbb{R}^{n\times n}$ được gọi là chéo hóa được (diagonalizable) nếu tồn tại ma trận $P$ khả nghịch và ma trận đường chéo $D$ sao cho:

$$A = PDP^{-1}$$

**Trong đó:**

- $D = \text{diag}(\lambda_1, \lambda_2, ..., \lambda_n)$ là ma trận đường chéo chứa các giá trị riêng (eigenvalues) của $A$ trên đường chéo chính.
- Các cột của ma trận $P$ là các vector riêng (eigenvectors) tương ứng với các giá trị riêng đó.

**Ứng dụng quan trọng:** Chéo hóa đóng vai trò cốt lõi trong tính toán khoa học, đặc biệt là giúp tính lũy thừa ma trận bậc cao cực kỳ nhanh chóng thông qua công thức $A^k = PD^kP^{-1}$. Việc này giúp giảm chi phí tính toán từ $O(n^3 \log k)$ xuống còn $O(n^2)$ sau khi đã thực hiện phân tích chéo hóa.

### 2.1.2. Điều kiện để chéo hóa Ma trận

Một ma trận vuông $A$ kích thước $n \times n$ chéo hóa được khi và chỉ khi thỏa mãn các điều kiện sau:

- Điều kiện cần và đủ: Ma trận $A$ có đủ $n$ vector riêng độc lập tuyến tính.
- Điều kiện đủ: Ma trận $A$ có $n$ giá trị riêng phân biệt.

### 2.1.3. Các bước giải tay (Manual Steps)

Để thực hiện chéo hóa ma trận $A$ bằng phương pháp thủ công, chúng ta tiến hành theo quy trình ba bước sau:

- **Bước 1:** Tìm giá trị riêng. Thực hiện giải phương trình đặc trưng $\det(A - \lambda I) = 0$ để xác định các nghiệm $\lambda_i$. Các nghiệm này chính là các giá trị riêng của ma trận.
- **Bước 2:** Tìm vector riêng. Với mỗi giá trị riêng $\lambda_i$ đã tìm được, tiến hành giải hệ phương trình tuyến tính thuần nhất $(A - \lambda_i I)v = 0$ để xác định không gian nghiệm. Các vector cơ sở của không gian này chính là các vector riêng tương ứng với $\lambda_i$.
- **Bước 3:** Lập ma trận $P$ và $D$. Nếu ma trận $A$ có đủ $n$ vector riêng độc lập tuyến tính, ta tiến hành xếp các vector này thành các cột của ma trận $P$. Đồng thời, lập ma trận đường chéo $D$ bằng cách xếp các giá trị riêng $\lambda_i$ tương ứng lên đường chéo chính.

## 2.2. Thiết kế và cài đặt thuật toán phân rã QR

Nhóm đã chọn quy trình chéo hóa bằng cách kết hợp phân rã **QR** và phép biến đổi **RREF** và đã cài đặt trong file `diagonalization.py`.

### 2.2.1. Phân rã QR bằng Gram-Schmidt

Để tìm giá trị riêng,hàm `qr_decomposition(A)` được cài đặt dựa trên thuật toán Gram-Schmidt cổ điển (CGS) nhằm trực chuẩn hóa các cột của ma trận $A$. Công thức xác định vector trực chuẩn $q_k$ tại mỗi bước được tính như sau:

$$q_k = \frac{a_k - \sum_{j=1}^{k-1} (a_k^T q_j) q_j}{\| a_k - \sum_{j=1}^{k-1} (a_k^T q_j) q_j \|}$$

Thông qua quy trình này, ma trận $A$ được phân rã thành tích của hai ma trận $Q$ và $R$, trong đó $Q$ là ma trận trực chuẩn ($Q^T Q = I$) và $R$ là ma trận tam giác trên. Đây là bước bổ trợ then chốt để triển khai thuật toán QR tìm giá trị riêng và trực quan hóa các bước thuật toán thông qua video Manim.

**Mã giả:**

**Input:** Ma trận A

**Output:** Ma trận Q, R

1. n ← số dòng của A
2. Cột_A ← các cột của A
3. Danh_sách_Q ← rỗng
4. Với i = 0 → n-1:
   - u_i ← Cột_A[i]
   - Với j = 0 → i-1: hệ_số ← ⟨Cột_A[i], Q[j]⟩ u_i ← u_i - hệ_số · Q[j]
   - độ_dài ← ||u_i||
   - Nếu độ_dài > ε: Thêm (u_i / độ_dài) vào Danh_sách_Q
   - Ngược lại: Thêm vector 0
5. Q ← chuyển vị của Danh_sách_Q
6. R ← Q^T A
7. Trả về Q, R

### 2.2.2. Thuật toán QR tìm giá trị riêng

Hàm `tim_tri_rieng_nxn(A)` thực hiện xác định các giá trị riêng thông qua một quy trình lặp hội tụ. Tại mỗi bước lặp $k$, thuật toán thực hiện hai thao tác cốt lõi:

- Phân rã: Thực hiện phân rã ma trận hiện tại thành $A_k = Q_k R_k$ bằng thuật toán Gram-Schmidt đã cài đặt.
- Cập nhật: Tính toán ma trận cho bước kế tiếp bằng cách nhân ngược lại các thành phần vừa phân rã: $A_{\{k+1\}} = R_k Q_k$.

Sau một số lượng vòng lặp đủ lớn (mặc định là 1000 bước trong mã nguồn), ma trận sẽ hội tụ về dạng tam giác trên. Khi đó, các giá trị nằm trên đường chéo chính của ma trận hội tụ này chính là tập hợp các giá trị riêng $\lambda_i$ của ma trận $A$ ban đầu.

**Mã giả:**

**Input:** A, max_iter, tol

**Output:** Các trị riêng của A

1. A_k ← A
2. Với k = 1 → max_iter:
   - (Q, R) ← qr_decomposition(A_k)
   - A_k ← RQ
3. Trả về diag(A_k)

### 2.2.3. Giải vector riêng bằng RREF

Sau khi đã xác định được tập hợp giá trị riêng $\lambda$, hàm tim_vector_rieng(A, lamda) sẽ tiến hành tìm các vector riêng tương ứng thông qua các bước sau:

Thiết lập ma trận: Tạo ma trận trung gian $M = A - \lambda I$.

Biến đổi RREF: Sử dụng hàm rref(M) đã xây dựng để đưa ma trận $M$ về dạng bậc thang rút gọn.Trích xuất không gian nghiệm: Nhóm tiến hành xác định các biến tự do (free variables) từ các cột không chứa phần tử chốt (pivot). Từ các biến này, thuật toán trích xuất các vector cơ sở của không gian nghiệm (Null Space), đây chính là các vector riêng cần tìm để phục vụ cho quá trình chéo hóa.

**Mã giả:**

**Input:** Ma trận A, giá trị riêng λ

**Output:** Vector riêng v

1. M ← A - λI
2. (M_rref, pivot_cols) ← rref(M)
3. Chọn cột tự do j
4. v ← 0
5. Nếu tồn tại j:
   - v[j] ← 1
   - Với mỗi pivot i: v[i] ← -M_rref[i][j]
6. Trả về v

### 2.2.4. Hàm chéo hóa tổng quát

Lắp ghép các giá trị riêng và vector riêng thành cấu trúc $A = PDP^{-1}$

**Mã giả:**

**Input:** Ma trận A, các trị riêng $\lambda$

**Output:** P, D

1. n ← kích thước của A
2. D ← ma trận đường chéo với các phần tử $\lambda_i$
3. Danh_sách_V ← rỗng
4. Với mỗi λ ∈ danh_sách$\lambda$:
   - v ← tim_vector_rieng(A, $\lambda$)
   - Thêm v vào Danh_sách_V
5. P ← ma trận có các cột là các vector trong Danh_sách_V
6. Trả về P, D

### 2.2.5. Kết quả thực nghiệm

Nhóm thực hiện kiểm chứng toàn bộ quy trình chéo hóa thủ công trên ma trận như sau:

**1. Nhóm Ma Trận Thông Thường (Kích thước $2 \times 2$)**

**Test Case 1: Ma trận $2 \times 2$**

Ma trận $A$:

$$\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

- Giá trị riêng: $[3.0, 1.0]$
- Ma trận $D$: $\text{diag}(3.0, 1.0)$
- Ma trận $P$: $\begin{pmatrix} 1.0 & -1.0 \\ 1.0 & 1.0 \end{pmatrix}$

**2. Nhóm Ma Trận Đặc Biệt**

**Test Case 2: Ma trận suy biến**

Ma trận $A$:

$$\begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}$$

- Giá trị riêng: $[5.0, 0.0]$ (do $\det(A) = 0$)

**Test Case 3: Ma trận đơn vị**

Ma trận $A$:

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

- Giá trị riêng: $[1.0, 1.0, 1.0]$
- Ma trận $P$:

$$\begin{pmatrix} 1.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & 0.0 \\ 0.0 & 0.0 & 1.0 \end{pmatrix}$$

- (Ma trận đơn vị)

**Test Case 4: Ma trận Hilbert $H_3$**

Ma trận $A$:

$$\begin{pmatrix} 1 & 0.5 & 0.3333 \\ 0.5 & 0.3333 & 0.25 \\ 0.3333 & 0.25 & 0.2 \end{pmatrix}$$

- Giá trị riêng: $[1.4083, 0.1223, 0.0027]$

**Test Case 5: Ma trận có giá trị riêng lặp**

Ma trận $A$:

$$\begin{pmatrix} 3 & 1 \\ 0 & 3 \end{pmatrix}$$

- Giá trị riêng: $[3.0, 3.0]$

**Đánh giá độ chính xác:**

- Để đảm bảo tính đúng đắn, nhóm đã sử dụng thư viện **NumPy** để kiểm chứng đẳng thức $AP = PD$. Sai số dư (Residual Error) thu được đạt ngưỡng xấp xỉ $1.11 \times 10^{-16}$, tiệm cận giới hạn sai số máy tính.
- Kết quả này khẳng định các thuật toán tự cài đặt thủ công từ phân rã QR đến biến đổi RREF đều hoạt động với độ chính xác cực kỳ cao, đáp ứng hoàn hảo yêu cầu kỹ thuật của đồ án.

## 2.3. Kỹ Thuật Phân Rã SVD

### 2.3.1. Cơ sở toán học

Với ma trận thực $A \in \mathbb{R}^{m\times n}$, phân tích SVD có dạng:

$$A = U\Sigma V^T,$$

trong đó:

1. $U \in \mathbb{R}^{m\times m}$ là ma trận trực giao ($U^TU = I_m$),
2. $V \in \mathbb{R}^{n\times n}$ là ma trận trực giao ($V^TV = I_n$),
3. $\Sigma \in \mathbb{R}^{m\times n}$ là ma trận đường chéo chữ nhật chứa các giá trị suy biến không âm $\sigma_i$ trên đường chéo chính.

Thuật toán khai thác tính chất phổ của ma trận đối xứng xác định nửa dương $A^TA$:

$$A^TA = V\Lambda V^T,$$

$$\Lambda = diag(\lambda_1, ..., \lambda_n),$$

$$\sigma_i = \sqrt{\max(0, \lambda_i)}.$$

Sau khi xác định $V$ và $\sigma_i$, các cột của $U$ được suy ra theo:

$$\boldsymbol{u}_i = \frac{A\boldsymbol{v}_i}{\sigma_i}, \quad (\sigma_i > 0).$$

### 2.3.2. Thiết kế và cài đặt thuật toán phân rã SVD

Chương trình trong file **decomposition.py** được xây dựng nhằm hiện thực hóa phương pháp phân tích giá trị suy biến cho ma trận thực. Các chức năng cốt lõi gồm:

1. Cài đặt thuật toán **phân tích giá trị suy biến (Singular Value Decomposition - SVD)** theo hướng minh bạch từng bước:
$$A = U\Sigma V^T.$$
2. Đối chiếu kết quả với **numpy.linalg.svd** để đánh giá độ chính xác số.
3. Thực nghiệm trên nhiều lớp ma trận (ma trận không, ma trận đơn vị, ma trận suy giảm hạng, ma trận chữ nhật và dữ liệu lệch thang đo).

File **manim_scene.py** được sử dụng để trực quan hóa ý nghĩa hình học của SVD theo chuỗi biến đổi xoay - co giãn - xoay, qua đó liên hệ giữa biểu thức đại số và trực giác hình học.

### 2.3.3. Kiến trúc thuật toán trong chương trình

Nhằm đảm bảo tính minh bạch của quy trình tính toán, chương trình hiện thực thủ công các phép toán nền tảng thay vì phụ thuộc hoàn toàn vào thư viện mức cao:

1. Tạo ma trận: **zeros_matrix**, **identity_matrix**.
2. Biến đổi ma trận/vector: **transpose**, **get_column**, **scale_vector**, **subtract_vectors**.
3. Phép toán số học: **dot_product**, **vector_norm**, **matrix_multiply**.

Cách tổ chức này phù hợp với mục tiêu học thuật, cho phép theo dõi và kiểm chứng từng bước trung gian của thuật toán.

- **Bước 1**: Giá trị riêng/eigenvector của $A^T A$

Trong hàm `svd_decomposition`, chương trình sử dụng cơ chế lai để cân bằng giữa tính diễn giải và hiệu năng tính toán:

1. Với ma trận bậc nhỏ ($n \leq 4$): áp dụng **QR iteration tự cài đặt**.
2. Với ma trận bậc lớn ($n > 4$): sử dụng `numpy.linalg.eigh` để cải thiện độ ổn định số và tốc độ hội tụ.

**QR iteration tự cài đặt:**

Thuật toán `qr_eigen_decomposition` được triển khai theo các bước:

1. **modified_gram_schmidt(A_k)** để tách $A_k = QR$,
2. cập nhật $A_{k+1} = RQ$,
3. tích lũy ma trận riêng $Q_{\text{total}} \leftarrow Q_{\text{total}} Q$,
4. dừng khi tổng bình phương phần tử ngoài đường chéo nhỏ hơn ngưỡng **tol**.

Đây là dạng hiện thực gần với quy trình tính tay trị riêng của ma trận đối xứng.

- **Bước 2**: Singular values và ma trận $\Sigma$

Từ các trị riêng $\lambda_i$, chương trình xác định các giá trị suy biến theo:

$$\sigma_i = \sqrt{\max(0, \lambda_i)}$$

Biểu thức $\max(0, \lambda_i)$ giúp triệt tiêu sai lệch âm rất nhỏ phát sinh do sai số dấu phẩy động.

Ma trận $\Sigma$ được xây dựng với kích thước $m \times n$, trong đó các phần tử đường chéo chính đến chỉ số $\min(m, n)$ nhận giá trị $\sigma_i$.

- **Bước 3:** Dựng ma trận $U$

Với mỗi cặp $(\sigma_i, \boldsymbol{v}_i)$, chương trình xử lý như sau:

1. Nếu $\sigma_i > 10^{-12}$: tính $\boldsymbol{u}_i = A\frac{\boldsymbol{v}_i}{\sigma_i}$.
2. Nếu $\sigma_i \approx 0$: tạm thời đặt cột 0.

Khi số cột thu được chưa đủ để tạo ma trận $U$ kích thước $m \times m$, chương trình bổ sung các vector trực chuẩn thông qua Gram-Schmidt từ vector ngẫu nhiên. Bước này đảm bảo đầu ra ở chế độ **full_matrices**.

- **Bước 4**: Tạo $V^T$ và trả kết quả

  Từ ma trận $V$ (mỗi cột là một eigenvector của $A^TA$), hàm trả về:

$$(U, \Sigma, V^T, \text{singular_values}).$$

### 2.3.4. Đánh giá

#### 2.3.4.1. Mã giả tổng quát

**Input:** Ma trận A $\in \mathbb{R}^{m\times n}$

**Output:** U, Σ, $V^T$

1. AtA ← $A^T$A
2. Nếu n > 4:
   - (λ, V) ← eigh(AtA)

   Ngược lại:
   - (λ, V) ← QR_iteration(AtA)
3. Sắp xếp (λ, V) theo λ giảm dần
4. σ_i ← sqrt(max(0, λ_i)) Xây dựng ma trận Σ
5. Với i = 1 → min(m, n):
   - Nếu σ_i > ε: u_i ← (A v_i) / σ_i
   - Ngược lại: u_i ← 0
6. Hoàn thiện U thành ma trận trực chuẩn (m × m) nếu cần
7. $V^T$ ← chuyển vị của V
8. Trả về U, Σ, $V^T$

#### 2.3.4.2. Độ phức tạp tính toán

Giả sử đầu vào có kích thước $m \times n$, các thành phần chi phí chính gồm:

1. Tính $A^T A$: $O(mn^2)$.
2. QR iteration (nếu dùng bản custom): mỗi vòng xấp xỉ $O(n^3)$, tổng là $O(Tn^3)$ với $T$ số vòng lặp.
3. Dựng $U$: khoảng $O(mn \min(m, n))$.

Nhìn chung, bước phân tích trị riêng của $A^T A$ là thành phần chi phối độ phức tạp toàn thuật toán.

#### 2.3.4.3. Kiểm chứng kết quả trong chương trình

Hàm `check_result` đánh giá chất lượng nghiệm bằng cách so sánh kết quả custom với kết quả tham chiếu từ **NumPy** theo ba chỉ số:

1. Lỗi tái tạo custom:
$$\|A - U\Sigma V^T\|_F.$$
2. Lỗi tái tạo NumPy:
$$\|A - U_{\text{np}}\Sigma_{\text{np}}V_{\text{np}}^T\|_F.$$
3. Sai khác singular values:
$$\|s_{\text{custom}} - s_{\text{np}}\|_2.$$

Các chỉ số trên cho phép khẳng định:

1. Tính đúng đắn toán học.
2. Mức độ tiệm cận giữa nghiệm thủ công và nghiệm chuẩn thư viện.
3. Độ ổn định trên các trường hợp biên (rank thiếu, cột 0, lệch thang đo lớn).

## 2.4. Trực Quan Hóa với Manim

Phần này trình bày kết quả trực quan hóa hình học của các phép biến đổi ma trận thông qua video demo (**demo_video.mp4**). Video tập trung vào hai kỹ thuật phân tách quan trọng là **SVD** và **Chéo hóa**, áp dụng cho ma trận không đối xứng:

$$\boldsymbol{A} = \begin{pmatrix} 5 & 0 \\ 4 & 3 \end{pmatrix}$$

### 2.4.1. Phân rã Giá trị Kỳ dị (SVD) từng bước

Video minh họa quá trình tính toán SVD ($\boldsymbol{A} = \boldsymbol{U\Sigma V^T}$) một cách chi tiết:

- **Tính toán $\boldsymbol{A^T A}$**: Xác định ma trận đối xứng để tìm trị riêng và vector riêng.

- **Trị riêng và Giá trị kỳ dị**: Giải phương trình đặc trưng để tìm $\lambda_1 = 45, \lambda_2 = 5$, từ đó suy ra các giá trị kỳ dị $\sigma_1 = 3\sqrt{5}$ và $\sigma_2 = \sqrt{5}$.
- **Xây dựng các ma trận**: Minh họa cách gộp các vector riêng của $\boldsymbol{A^T A}$ thành ma trận $\boldsymbol{V}$ và tính toán các cột của $\boldsymbol{U}$ qua công thức $(u)_i = \frac{\boldsymbol{A}(v)_i}{\sigma_i}$.

### 2.4.2. Trực quan hóa Biến đổi Hình học

Sử dụng hình ảnh hình tròn đơn vị biến đổi thành hình Elip để giải thích ý nghĩa hình học của SVD qua 3 pha:

- **Pha 1 (Xoay - $\boldsymbol{V^T}$)**: Xoay các vector cơ sở $v_1, v_2$ về các trục chuẩn $e_1, e_2$.
- **Pha 2 (Co giãn - $\boldsymbol{\Sigma}$)**: Kéo giãn không gian dọc theo các trục chuẩn với hệ số $\sigma_1$ và $\sigma_2$.
- **Pha 3 (Xoay - $\boldsymbol{U}$)**: Xoay Elip về vị trí cuối cùng trong không gian đích. Quá trình này giúp hiểu rõ cách một ma trận ma trận tác động lên mọi vector trong không gian.

### 2.4.3. Quá trình thực hiện Chéo hóa (Diagonalization)

Video minh họa các bước thuật toán để phân rã ma trận $\boldsymbol{A}$ về dạng chéo $\boldsymbol{A} = \boldsymbol{PDP^{-1}}$:

- **Bước 1: Giải phương trình đặc trưng:** Video bắt đầu bằng việc tính $\det(\boldsymbol{A} - \lambda \boldsymbol{I}) = 0$. Quá trình khai triển đa thức đặc trưng $\lambda^2 - 8\lambda + 15 = 0$ giúp xác định hai trị riêng thực phân biệt là $\lambda_1 = 5$ và $\lambda_2 = 3$.
- **Bước 2: Tìm các vector riêng:** Với mỗi trị riêng, video thực hiện giải hệ phương trình $(\boldsymbol{A} - \lambda \boldsymbol{I})(v) = (0)$. Kết quả tìm được hai vector riêng tương ứng là $(v)_1 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$ và $(v)_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$. Đây chính là các hướng mà không gian chỉ bị co giãn chứ không bị quay khi nhân với $\boldsymbol{A}$.
- **Bước 3: Xây dựng ma trận $\boldsymbol{P}$ và $\boldsymbol{D}$:** Các vector riêng được ghép thành các cột của ma trận chuyển cơ sở $\boldsymbol{P} = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}$, và các trị riêng được đặt trên đường chéo chính của ma trận $\boldsymbol{D} = \begin{pmatrix} 5 & 0 \\ 0 & 3 \end{pmatrix}$.
- **Bước 4: Kiểm chứng và Nghịch đảo:** Cuối cùng, video tính toán $\boldsymbol{P^{-1}} = \begin{pmatrix} 1 & 0 \\ -2 & 1 \end{pmatrix}$ và hiển thị phép nhân hoàn chỉnh $\boldsymbol{A} = \boldsymbol{PDP^{-1}}$.

## 2.5. Cài đặt và Kết quả thực nghiệm

### 2.5.1. Tổ chức cấu trúc dữ liệu

Nhóm sử dụng cấu trúc danh sách hai chiều `List[List[float]]` của Python thuần. Ngưỡng sai số (epsilon) được đặt ở mức $10^{-12}$ để so sánh các số thực với 0. Các hàm xử lý được chia nhỏ thành các file module riêng biệt (`gaussian.py`, `determinant.py`, `inverse.py`…).

### 2.5.2. Đánh giá độ chính xác và Sức chịu tải (Stress Test)

Nhóm đã kiểm thử trên ma trận ngẫu nhiên kích thước $100 \times 100$. Kết quả kiểm chứng phần dư (Residual Error) bằng thư viện NumPy cho thấy:

$$\|Ax - b\|_\infty < 10^{-8}$$

Thuật toán xử lý xuất sắc cả 3 trường hợp: Nghiệm duy nhất, Hệ vô nghiệm, và Hệ có vô số nghiệm. Điều này khẳng định thuật toán không bị nổ sai số dồn tích khi thực hiện hàng chục ngàn phép tính.

## 2.6. Tiêu chí đánh giá — Phần 2

| **Tiêu chí** | **Mô tả** | **Điểm** |
|---|---|---|
| Lý thuyết | Viết mô tả về phân rã SVD và điều kiện chéo hóa chính xác | 0.5 |
| Cài đặt SVD | Cài đặt chương trình phân rã SVD dựa trên phương pháp tính tay. Sử dụng QR để hỗ trợ tìm eigenvalues | 1.5 |
| Chéo hóa | Tìm đúng vector riêng và ma trận $\boldsymbol{P}, \boldsymbol{D}$ | 0.5 |
| Video Manim | Hoạt ảnh minh họa rõ bản chất hình học của $\lambda$ và $\boldsymbol{v}$ | 2.5 |
| Điểm cộng | Cài đặt phân rã bổ sung (SVD) | 1.0 |
| **Tổng Phần 2** | **Đã hoàn thành 100%** | **6.0** |

# 3. Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng

**Tóm tắt yêu cầu Phần 3**

So sánh các phương pháp giải $\mathbf{Ax} = \mathbf{b}$, bao gồm Gauss (Phần 1), phân rã (Phần 2) và ít nhất một phương pháp lặp. Phân tích tính ổn định số và chi phí tính toán qua thực nghiệm.

## 3.1. Giới thiệu

Phần 3 của đồ án tập trung vào **giải hệ phương trình tuyến tính** $Ax = b$ bằng ba phương pháp khác nhau, đồng thời thực hiện **đánh giá hiệu năng** (thời gian thực thi) và **phân tích tính ổn định số** (sai số tương đối) của từng phương pháp.

### 3.1.1. Mục tiêu cụ thể

- Cài đặt và tích hợp ba thuật toán giải hệ: **Khử Gauss** (Part 1), **SVD** (Part 2), và **Gauss-Seidel** (Part 3).
- Đo thời gian thực thi trên các ma trận SPD kích thước $n \in \{50, 100, 200, 500, 1000\}$.
- Phân tích độ phức tạp $O(n^3)$ thông qua đồ thị Log-Log.
- Đánh giá tính ổn định số trên ma trận Hilbert (ill-conditioned) và ma trận SPD ngẫu nhiên (well-conditioned).

### 3.1.2. Cấu trúc thư mục

```
part3/
├── solvers.py        # Module chứa 3 thuật toán giải hệ
├── benchmark.py      # Module đo hiệu năng và tính sai sô´
├── analysis.ipynb    # Notebook phân tích và trực quan hóa
└── report_part3.typ  # Báo cáo
```

## 3.2. Cơ sở lý thuyết

### 3.2.1. Số điều kiện (Condition Number)

Số điều kiện của ma trận $A$ theo chuẩn $p$ được định nghĩa:

$$\kappa_p(A) = \|A\|_p \cdot \left\|A^{-1}\right\|_p$$

**Ý nghĩa:** Số điều kiện đo lường độ nhạy cảm của nghiệm $x$ đối với nhiễu trong dữ liệu đầu vào. Cụ thể, nếu $Ax = b$ và $A\hat{x} = b + \delta b$, ta có:

$$\frac{\|\hat{x} - x\|_2}{\|x\|_2} \leq \kappa(A) \cdot \frac{\|\delta b\|_2}{\|b\|_2}$$

- $\kappa(A) \approx 1$: Ma trận **well-conditioned** (ổn định tốt).
- $\kappa(A) \gg 1$: Ma trận **ill-conditioned** (sai số bị khuếch đại).

### 3.2.2. Phương pháp Khử Gauss với Partial Pivoting

**Ý tưởng:** Biến đổi hệ $Ax = b$ về dạng tam giác trên $Ux = c$, sau đó thế ngược để tìm $x$. Tại mỗi bước khử, hoán đổi hàng để đưa phần tử có trị tuyệt đối lớn nhất lên vị trí pivot.

**Độ phức tạp:** $O\left(\frac{2n^3}{3}\right) \approx O(n^3)$

### 3.2.3. Phương pháp SVD (Singular Value Decomposition)

**Phân rã:** Mọi ma trận $A \in \mathbb{R}^{m \times n}$ đều có thể phân rã thành $A = U\Sigma V^T$. Nghiệm của hệ được tính qua:

$$x = V\Sigma^{-1}U^T b$$

.

**Ưu điểm:** Ổn định số rất cao, có thể phát hiện và xử lý ma trận suy biến thông qua việc triệt tiêu các giá trị kỳ dị nhỏ.

### 3.2.4. Phương pháp lặp Gauss-Seidel

**Ý tưởng:** Lặp cập nhật từng thành phần $x_i$ theo công thức:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left( b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(k)} \right)$$

**Điều kiện hội tụ:** Ma trận $A$ **chéo trội chặt hàng** hoặc **đối xứng xác định dương** (SPD).

**Tiêu chí dừng:** $\left\| x^{(k+1)} - x^{(k)} \right\|_\infty < \epsilon$ với $\epsilon = 10^{-10}$.

### 3.2.5. Bảng so sánh tổng quan

| **Tiêu chí** | **Khử Gauss (PP)** | **SVD** | **Gauss-Seidel** |
|---|---|---|---|
| Loại | Trực tiếp | Trực tiếp | Lặp |

| Tiêu chí | Khử Gauss (PP) | SVD | Gauss-Seidel |
|---|---|---|---|
| Độ phức tạp | $O(n^3)$ | $O(n^3)$ | $O(kn^2)$ |
| Độ ổn định | Cao | Rất cao | Trung bình |
| Yêu cầu ma trận | Không suy biến | Bất kỳ | SPD / Chéo trội |

## 3.3. Phương pháp thực nghiệm

### 3.3.1. Ma trận thí nghiệm

- **Ma trận SPD ngẫu nhiên:**

$$A = B^T B + nI_n$$

. Đảm bảo Gauss-Seidel hội tụ.

- **Ma trận Hilbert:**

$$H_{ij} = 1/(i + j - 1)$$

. Ví dụ điển hình về sự mất ổn định số.

### 3.3.2. Phương pháp đo

- **Thời gian:** Trung bình 5 lần chạy cho mỗi cặp $(n, \text{method})$.
- **Sai số tương đối:**

$$\text{err} = \frac{\|Ax - b\|_2}{\|b\|_2}$$

.

## 3.4. Phân tích hiệu năng (Performance Analysis)

### 3.4.1. Kết quả đo thời gian thực thi

| $n$ | Gauss (s) | SVD (s) | Gauss-Seidel (s) |
|---|---|---|---|
| 50 | 0.0022 | 0.0006 | 0.0022 |
| 100 | 0.0088 | 0.0044 | 0.0043 |
| 200 | 0.0357 | 0.0115 | 0.0098 |
| 500 | 0.2238 | 0.1012 | 0.0285 |
| 1000 | 6.4125 | 2.1184 | 0.4604 |

### 3.4.2. Phân tích đồ thị Log-Log và Độ phức tạp

- **Khử Gauss và SVD**: Xác nhận độ phức tạp lý thuyết $O(n^3)$ với hệ số hồi quy $\alpha \approx 3$.
- **Gauss-Seidel**: Độ dốc thấp hơn ($\approx 2.2$), cho thấy ưu thế vượt trội về tốc độ trên hệ SPD lớn (nhanh hơn Gauss  14 lần tại $n = 1000$).

## 3.5. Phân tích tính ổn định số (Numerical Stability)

### 3.5.1. Kết quả trên ma trận Hilbert (Ill-conditioned)

| n | Gauss Error | SVD Error | GS Error | $\kappa(H_n)$ |
|---|---|---|---|---|
| 3 | $1.09 \times 10^{-16}$ | 1.71e+05 | $1.30 \times 10^{-11}$ | $5.24 \times 10^{2}$ |
| 5 | $1.66 \times 10^{-16}$ | $1.94 \times 10^{-11}$ | $2.77 \times 10^{-6}$ | $4.77 \times 10^{5}$ |
| 7 | 0.00 | $1.45 \times 10^{-10}$ | $1.08 \times 10^{-6}$ | $4.75 \times 10^{8}$ |
| 10 | $3.33 \times 10^{-17}$ | $2.39 \times 10^{-7}$ | $4.19 \times 10^{-6}$ | $1.60 \times 10^{13}$ |
| 12 | N/A (Failed) | $1.39 \times 10^{-9}$ | $4.23 \times 10^{-6}$ | $1.76 \times 10^{16}$ |

### 3.5.2. Kết quả trên ma trận SPD ngẫu nhiên (Well-conditioned)

| n | Gauss Error | SVD Error | GS Error |
|---|---|---|---|
| 50 | $3.44 \times 10^{-16}$ | $2.05 \times 10^{-15}$ | $2.46 \times 10^{-12}$ |
| 100 | $4.23 \times 10^{-16}$ | $2.58 \times 10^{-15}$ | $4.62 \times 10^{-12}$ |
| 200 | $5.92 \times 10^{-16}$ | $2.98 \times 10^{-15}$ | $6.08 \times 10^{-12}$ |
| 500 | $9.15 \times 10^{-16}$ | $2.88 \times 10^{-15}$ | $4.47 \times 10^{-12}$ |

### 3.5.3. Nhận xét và phân tích

- **Độ chính xác trên hệ SPD:** Cả ba phương pháp đều đạt độ chính xác cực cao ($10^{-12}$ đến $10^{-16}$). Trong đó Gauss và SVD cho kết quả gần với sai số máy tính ($\epsilon_{\text{machine}}$), còn Gauss-Seidel dừng lại ở ngưỡng sai số hội tụ ($10^{-10}$ đến $10^{-12}$).
- **Tính ổn định trên hệ Hilbert:**
  - **SVD (An toàn nhất):** Duy trì sai số ổn định ngay cả khi số điều kiện tiệm cận $10^{16}$. Đây là phương pháp "cứu cánh" cho các ma trận cực kỳ kém điều kiện.
  - **Gauss (Chính xác cao):** Rất hiệu quả nhờ Partial Pivoting nhưng bắt đầu mất ổn định và thất bại hoàn toàn khi số điều kiện vượt quá $10^{16}$ ($n \geq 12$).

- **Gauss-Seidel (Hạn chế):** Thất bại trong việc hội tụ trên ma trận Hilbert do ma trận này không thỏa điều kiện chéo trội và có cấu trúc quá phức tạp cho các phương pháp lặp đơn giản.

### 3.5.4. Mối quan hệ $\kappa(A)$ và sai số

Kết quả thực nghiệm xác nhận bất đẳng thức lý thuyết:

$$\frac{\|\hat{x} - x\|_2}{\|x\|_2} \leq \kappa(A) \cdot \epsilon_{\text{machine}}$$

Khi $\kappa(A)$ tiệm cận $10^{16}$, sự ổn định số học bị đe dọa nghiêm trọng, dẫn đến việc phương pháp trực tiếp như Gauss-PP thất bại trong việc tìm nghiệm chính xác.

## 3.6. Kết luận và Khuyến nghị

1. Sử dụng **Gauss-Seidel** cho hệ SPD lớn để tối ưu tốc độ.
2. Sử dụng **SVD** cho các bài toán có nguy cơ suy biến cao hoặc dữ liệu nhiễu.
3. Kỹ thuật **Vectorization** là yếu tố sống còn khi thực thi trên Python.

## 3.7. Tiêu chí đánh giá — Phần 3

| Tiêu chí | Mô tả | Điểm |
|---|---|---|
| Cài đặt phương pháp lặp | Viết mô tả về phân rã SVD và điều kiện chéo hóa chính xác | 0.5 |
| Thực nghiệm thời gian | Cài đặt chương trình phân rã SVD dựa trên phương pháp tính tay. Sử dụng QR để hỗ trợ tìm eigenvalues | 1.5 |
| Phân tích ổn định số | Tìm đúng vector riêng và ma trận $\boldsymbol{P}, \boldsymbol{D}$ | 0.5 |
| Nhận xét và kết luận | Hoạt ảnh minh họa rõ bản chất hình học của $\lambda$ và $\boldsymbol{v}$ | 2.5 |
| Trình bày Notebook | Cài đặt phân rã bổ sung (SVD) | 1.0 |
| **Tổng Phần 3** | **Đã hoàn thành 100%** | **6.0** |

# 4. Kết luận

## 4.1. Tóm tắt kết quả đạt được

Nhóm đã hoàn thành 100% các yêu cầu đặt ra. Đã xây dựng thành công bộ thư viện mini giải quyết các bài toán đại số tuyến tính cơ bản, xử lý tốt sai số bằng Partial Pivoting và xuất ra được nghiệm tổng quát cho hệ vô số nghiệm. Mã nguồn được tổ chức sạch sẽ, chuẩn hóa đầu ra.

## 4.2. Bài học rút ra

Thông qua đồ án, nhóm đã rèn luyện được tư duy chuyển đổi từ công thức toán học hàn lâm sang các dòng code thực tế. Nhóm hiểu sâu hơn về kiến trúc máy tính, đặc biệt là cách máy tính biểu diễn số thực (Floating point) và giới hạn của việc làm tròn số trong tính toán khoa học.

## 4.3. Khó khăn gặp phải

Khó khăn lớn nhất là việc xử lý trường hợp hệ phương trình có vô số nghiệm. Việc truy vết các cột không chứa phần tử giải (non-pivot columns) để xác định biến tự do (free variables) và xây dựng cơ sở không gian nghiệm (nullspace basis) đòi hỏi cấu trúc vòng lặp phức tạp và dễ gây lỗi mảng (index out of bounds).

# 5. Bảng phân công công việc

| Họ và Tên | Nội dung công việc | Trạng thái | Trách nhiệm Report |
|---|---|---|---|
| Nguyễn Duy Vũ | Part 1: `inverse.py`, `rank_basis.py` và demo trong `part1_demo.ipynb` | **Hoàn thành** | Report Part 1 |
| Hà Lê Thanh Thúy | Part 1: `gaussian.py`, `determinant.py` và demo trong `part1_demo.ipynb` | **Hoàn thành** | Report Part 1 |
| Phạm Văn Hữu Tài | Part 2: `diagonalization.py` | **Hoàn thành** | Report Part 2 |
| Trần Minh Tiến | Part 2: `decomposition.py` | **Hoàn thành** | Report Part 2 |
| Nguyễn Hữu Tường Vân | Part 2: `manim_scene.py`, `demo_video.mp4` và Full Part 3 | **Hoàn thành** | Report Part 2 & 3 |

*Lưu ý: Mỗi thành viên chịu trách nhiệm viết nội dung report tương ứng với phần việc đã đảm nhiệm trên hệ thống.*

*Nhóm có sử dụng các công cụ AI hỗ trợ trong đồ án án bao gồm: (ChatGPT, Gemini, Copilot) và các phần mềm khác như (Visual studio code, Google Colab, Typst)*

# Tài Liệu Tham Khảo

[1] **Gilbert Strang**, *Introduction to Linear Algebra*, 6th Edition, Wellesley-Cambridge Press, 2023.
(Giáo trình gối đầu giường về Đại số tuyến tính, cung cấp cái nhìn trực quan về các phép biến đổi ma trận: math.mit.edu/gs/linearalgebra)

[2] **Lloyd N. Trefethen & David Bau III**, *Numerical Linear Algebra*, SIAM, 1997.
(Tài liệu chuyên sâu về các thuật toán số học như QR, SVD và tính ổn định của hệ phương trình.)

[3] **3Blue1Brown (Grant Sanderson)**, *Essence of Linear Algebra* (Video series), YouTube, 2016.
(Nguồn cảm hứng chính cho phần trực quan hóa hình học trong Manim: youtube.com/playlist?list=...)

[4] **Manim Community Developers**, *Manim Documentation*, v0.18, 2024.
(Tài liệu hướng dẫn cài đặt và sử dụng thư viện Manim để tạo hoạt ảnh toán học: docs.manim.community)

[5] **Python Software Foundation**, *The Python Language Reference*, v3.12, 2024.
(Tra cứu về cấu trúc dữ liệu cơ bản để cài đặt thuật toán từ đầu không dùng thư viện: docs.python.org/3)

[6] **Harris, C.R. et al.**, *Array programming with NumPy*, Nature, 2020.
(Sử dụng làm tiêu chuẩn để đối chiếu và kiểm chứng (verify) kết quả thuật toán tự cài đặt: numpy.org/doc/stable)

[7] **William H. Press et al.**, *Numerical Recipes: The Art of Scientific Computing*, 3rd Edition, Cambridge University Press, 2007.
(Cung cấp mã giả và phân tích chi tiết về hiệu năng/độ phức tạp của các thuật toán giải hệ phương trình.)

### Các nguồn tham khảo trực tuyến:

[8] **Robert Beezer**, *A First Course in Linear Algebra* (Version 3.5), 2015.
https://linear.ups.edu/ (Giáo trình mã nguồn mở đầy đủ lý thuyết về RREF và cơ sở không gian).

[9] **Cleve Moler**, *Numerical Computing with MATLAB*, SIAM, 2004.
mathworks.com/moler/chapters.html (Phần giải thích về sự khác biệt giữa các phương pháp giải hệ trực tiếp và lặp).